

LAPORAN PRAKTIKUM
TEKNOLOGI IOT II



Oleh :

Nama : LILI CAHYA PUTRI

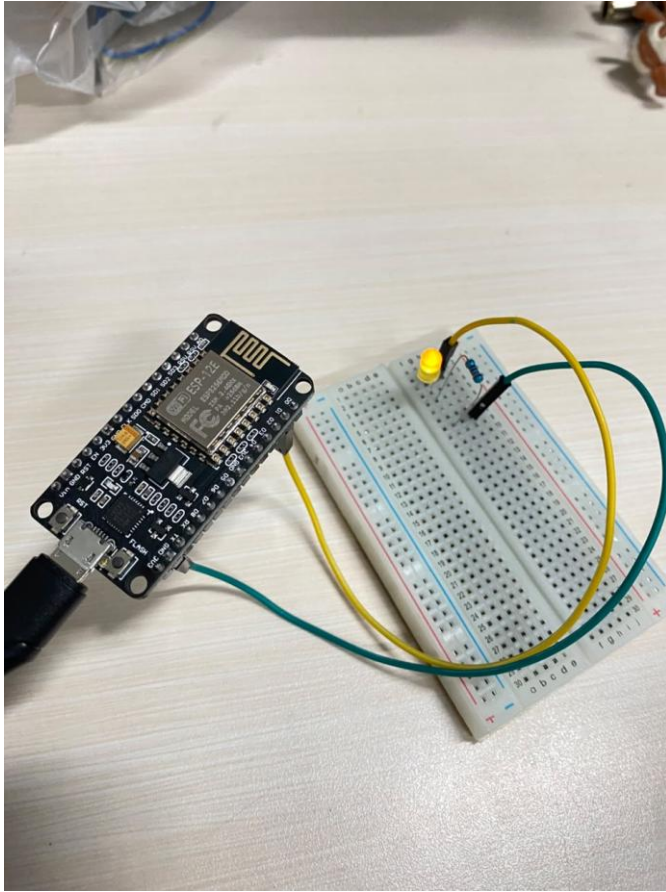
NPM : 25782071

Kelas : TRI 3C

PROGRAM STUDI TEKNOLOGI REKAYASA INTERNET
JURUSAN TEKNOLOGI INFORMASI
POLITEKNIK NEGERI LAMPUNG

2026

1.5.1 Praktikum 1: *Digital Output* (Menyalakan LED)



```
Blinkino
1  const int ledPin = 5;
2
3  void setup() {
4    Serial.begin(115200);
5    pinMode(ledPin, OUTPUT);
6    Serial.println("Praktikum 1 - Digital Output Dimulai!");
7  }
8
9  void loop() {
10   digitalWrite(ledPin, HIGH);
11   Serial.println("LED Menyala");
12   delay(1000);
13
14   digitalWrite(ledPin, LOW);
15   Serial.println("LED Mati");
16   delay(1000);
17 }
```

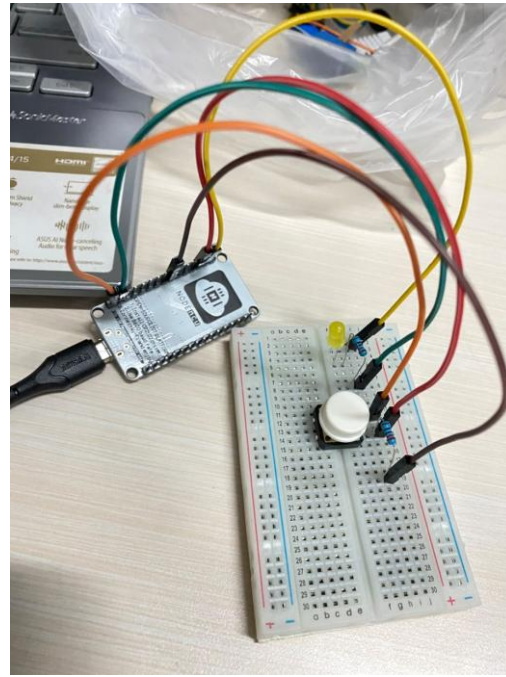
Output Serial Monitor X

Message [Enter to send message to 'NodeMCU 1.0 (ESP-12E Module)' on 'COM3'] New Line 115200 baud

LED Mati
LED Menyala
LED Mati
LED Menyala
LED Mati
LED Menyala
LED Mati

NodeMCU 1.0 (ESP-12E Module) on COM3 4:28 PM 9/14/2026

1.5.2 Praktikum 2: *Digital Input* (Membaca *Push Button*)



```
Blinkino
>
6 void setup() {
7   Serial.begin(115200);
8   pinMode(buttonPin, INPUT);
9   pinMode(ledPin, OUTPUT);
10 }
11
12 void loop() {
13   buttonState = digitalRead(buttonPin);
14
15   if (buttonState == HIGH) {
16     digitalWrite(ledPin, HIGH);
17     Serial.println("Tombol ditekan! -> LED ON");
18   } else {
19     digitalWrite(ledPin, LOW);
20   }
21 }
```

Output Serial Monitor X

Message (Enter to send message to 'NodeMCU 1.0 (ESP-12E Module)' on 'COM3')

New Line 115200 baud

Tombol ditekan! -> LED ON
Tombol ditekan! -> LED ON
Tombol ditekan! -> LED ON
Tombol ditekan! -> LED ON
Tombol ditekan! -> LED ON
Tombol ditekan! -> LED ON

NodeMCU 1.0 (ESP-12E Module) on COM3 5:17 PM 9/14/2026

1.6 Latihan

Latihan 1: Pemahaman Logika Pin

- Yang bakal terjadi: NodeMCU ESP8266 bakal gagal *booting* (ngadat/stuck dan nggak mau jalan).
- Penjelasannya: Menurut tabel pinout, pin D8 (GPIO 15) itu punya syarat khusus pas alat baru dinyalain: pinnya harus ditarik ke LOW (GND). Kalau tombolnya kamu tekan pas

banget NodeMCU baru dapet daya, arus 3.3V bakal masuk ke D8 (terbaca HIGH). Karena syarat booting-nya gak terpenuhi, sistem chip ESP8266-nya langsung *crash* atau *gagal boot*.

Latihan 2: Implementasi Kode

Dua baris fungsi `delay()` di fungsi `loop()` yang perlu kamu ganti cukup diubah angkanya aja:

```
delay(200); // Bikin LED nyala cepet banget (200 ms)
```

```
delay(800); // Bikin LED mati agak lama (800 ms)
```

Bentuk kode lengkap di `loop()`:

```
void loop() {  
    digitalWrite(ledPin, HIGH);  
    Serial.println("LED Menyala");  
    delay(200); // Nyala 200 ms  
  
    digitalWrite(ledPin, LOW);  
    Serial.println("LED Mati");  
    delay(800); // Mati 800 ms  
}
```

Latihan 3: Analisis Sirkuit

- **Apakah kodenya error pas di-compile?**

Enggak. Kodenya bakal tetap sukses di-*compile* dan di-*upload* tanpa error sama sekali. Soalnya yang salah itu di pemasangan komponen fisik (*hardware*), bukan di penulisan kodenya (*software*).

- **Kenapa LED-nya gak nyala?**

LED itu kan dioda (komponen polaritas), jadi arus DC cuma bisa mengalir dari **Anoda (kaki positif)** ke **Katoda (kaki negatif)**. Kalau dipasang terbalik (Anoda ke GND, Katoda ke D1), jalurnya kebalik (*reverse bias*). Efeknya, LED bakal menahan arus listrik yang lewat, jadinya LED gak bakal nyala sama sekali.

Latihan 4: Pengembangan Algoritma

Tinggal balik aja kondisi logika if-else di kodenya biar jadi terbalik (*Active-Low* secara software): Tinggal balik aja kondisi logika if-else di kodenya biar jadi terbalik (*Active-Low* secara software):

```

void loop() {

    buttonState = digitalRead(buttonPin);

    // Logikanya dibalik:

    if (buttonState == LOW) {

        digitalWrite(ledPin, HIGH); // Pas tombol LAGI NGGAK DITEKAN, LED malah NYALA

        Serial.println("Tombol tidak ditekan -> LED ON");

    } else {

        digitalWrite(ledPin, LOW); // Pas tombol DITEKAN, LED malah MATI

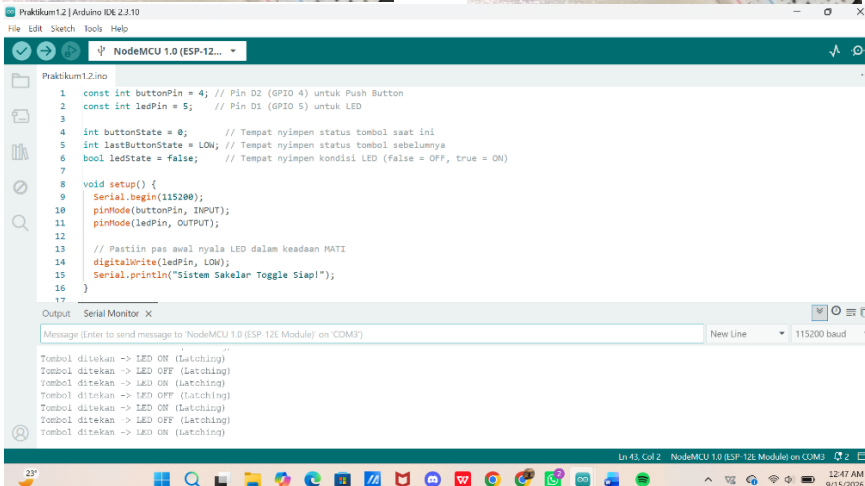
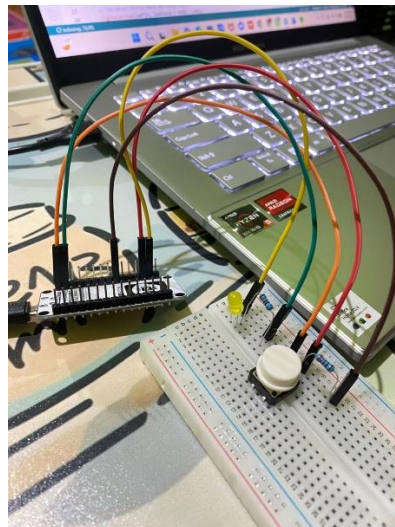
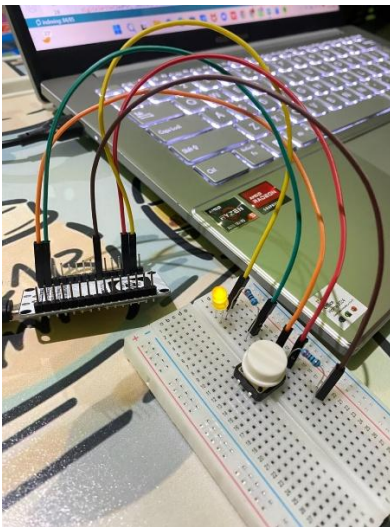
        Serial.println("Tombol ditekan! -> LED OFF");

    }

}

```

1.7 Tugas



Penjelasan Singkat

1. Rangkaian Hardware:

Tidak ada yang perlu diubah dari Praktikum 2. Tetap pakai sakelar di D2 dengan resistor *pull-down* 10k Ohm ke GND, dan LED di D1.

2. Logika *State Tracking* (Pelacakan Status):

Biar gak perlu nahan tombol terus-terusan, kita pakai variabel `ledState` (boolean). Setiap kali ditekan, kodenya bakal ngebalik statusnya (`ledState = !ledState`).

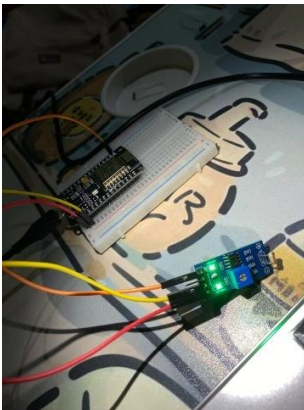
3. Logika *Edge Detection*:

Kondisi `if (buttonState == HIGH && lastButtonState == LOW)` dipake biar program cuma ngerespon **pas momen pertama kali tombol ditekan**, bukan pas tombol lagi ditahan lama.

4. Fungsi *Debouncing* (`delay(200)`):

Tombol fisik punya plat besi kecil di dalemnya yang suka "membal/bergetar" secara mikroskopis pas ditekan. `delay(200)` berfungsi ngasih jeda 200 milidetik biar getaran itu gak kebaca sebagai tekanan tombol berkali-kali secara tidak sengaja (*flickering*).

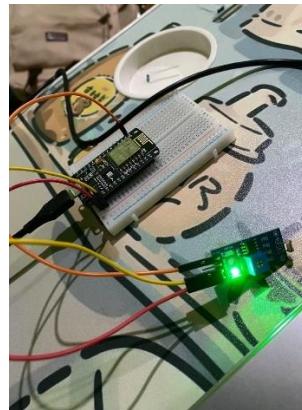
2.5.1 Praktikum 1: Pembacaan Intensitas Cahaya (LDR) via ADC



```
Praktikum17: Arduino-IDE (2.3.5)
File Edit Sketch Tools Help

NodeMCU 1.0 (ESP-12...)
Praktikum17.ino
1 const byte ledPin = A0;
2
3 void setup() {
4   Serial.begin(115200);
5 }
6
7 void loop() {
8   int ledValue = analogRead(A0);
9   Serial.print("Intensitas Cahaya (ADC): ");
10  Serial.println(ledValue);
11  delay(2000);
12 }
13
14
Output Serial Monitor x
Message (Enter to send message to NodeMCU 1.0 (ESP-12E Module) on COM4)
New Line 115200
Tebarsinar Cahaya (ADC) : 442
Tebarsinar Cahaya (ADC) : 464
Tebarsinar Cahaya (ADC) : 469
Tebarsinar Cahaya (ADC) : 473
Tebarsinar Cahaya (ADC) : 479
Tebarsinar Cahaya (ADC) : 481
Tebarsinar Cahaya (ADC) : 489
```

Pakai flash

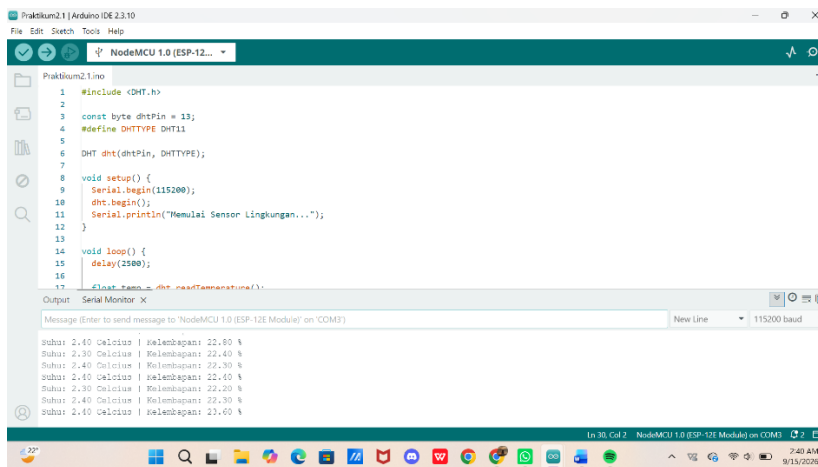


```
Praktikum17: Arduino-IDE (2.3.5)
File Edit Sketch Tools Help

NodeMCU 1.0 (ESP-12...)
Praktikum17.ino
1 const byte ledPin = A0;
2
3 void setup() {
4   Serial.begin(115200);
5 }
6
7 void loop() {
8   int ledValue = analogRead(A0);
9   Serial.print("Intensitas Cahaya (ADC): ");
10  Serial.println(ledValue);
11  delay(2000);
12 }
13
14
Output Serial Monitor x
Message (Enter to send message to NodeMCU 1.0 (ESP-12E Module) on COM4)
New Line 115200
Intensitas Cahaya (ADC) : 442
Intensitas Cahaya (ADC) : 455
Intensitas Cahaya (ADC) : 462
Intensitas Cahaya (ADC) : 473
Intensitas Cahaya (ADC) : 484
Intensitas Cahaya (ADC) : 495
```

Tanpa Flash

2.5.2 Praktikum 2: Membaca Data Sensor Suhu & Kelembapan (DHT)



2.6 Latihan

Latihan 1: Pemahaman Sistem ADC

- Jawaban: Soalnya ADC di NodeMCU ESP8266 itu resolusinya 10-bit.
- Penjelasan: Dalam dunia komputer (biner), 10-bit itu rumusnya $2^{10} = 1024$ kombinasi angka. Nah, karena hitungan komputer itu selalu dimulai dari angka 0, jadi rentang nilainya dibaca dari 0 sampai 1023. Makanya mentoknya di 1023, bukan 100 atau 1000.

Latihan 2: Implementasi Algoritma

- Rumus pembagian buat ngubah ke persentase (0% – 100%):

$$\text{Persentase} = \left(\frac{\text{Nilai ADC}}{1023.0} \right) \times 100$$

Tinggal ganti isi fungsi loop() di Arduino IDE jadi gini:

```
void loop() {  
  int ldrValue = analogRead(ldrPin);
```

```

    // Hitung persentase (pake 1023.0 biar hasilnya gak dibulatin ke 0)
    float persentase = (ldrValue / 1023.0) * 100;

    Serial.print("Intensitas Cahaya: ");
    Serial.print(persentase);
    Serial.println("%");

    delay(1000);
}

```

Latihan 3: Analisis Rangkaian

- **Yang bakal terjadi:** Lampu ruangan malah bakal **MATI**.
- **Penjelasan:** Soalnya modul Relay *Active-Low* itu logikanya kebalik. Relay baru bakal "ON" (nyambungin sakelar lampu) kalau dikasih sinyal **LOW (0V)**. Kalau program kamu ngirim sinyal **HIGH (3.3V)**, si relay malah menganggap itu perintah mati (OFF), jadinya lampu ruangan nggak bakal nyala.

Latihan 4: Pengembangan Logika

Gabungan pembacaan LDR sama pengontrolan LED di pin D1 (GPIO 5):

```

const byte ldrPin = A0;

const byte ledPin = 5; // Pin D1 (GPIO 5)

void setup() {
    Serial.begin(115200);

    pinMode(ledPin, OUTPUT);
}

void loop() {
    int ldrValue = analogRead(ldrPin);

    // LED nyala HANYA PAS kondisi gelap (nilai ADC di bawah 200)
    if (ldrValue < 200) {
        digitalWrite(ledPin, HIGH); // LED Menyala
        Serial.println("Gelap banget! LED -> ON");
    } else {
        digitalWrite(ledPin, LOW); // LED Mati
        Serial.println("Terang! LED -> OFF");
    }
}

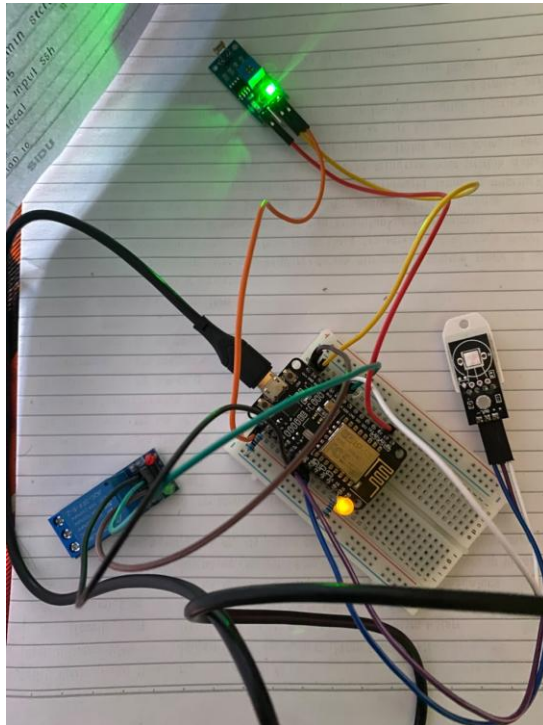
```

```
}
```

```
delay(1000);
```

```
}
```

2.7 Tugas: Sistem Otomasi Kontrol Lokal Terpadu



```
Praktikum2.1 | Arduino IDE 2.3.10
File Edit Sketch Tools Help

NodeMCU 1.0 (ESP-12E Module)

Praktikum2.1.ino
1 #include <DHT.h>
2
3 // Deklarasi Pin Fisik NodeMCU
4 const byte ldrPin = A0; // Pin A0 untuk LDR
5 const byte dhtPin = D5; // Pin D5 untuk DHT
6 const byte relayPin = D6; // Pin D6 untuk Relay
7 const byte ledPin = D1; // Pin D1 untuk LED Indikator
8
9 // Tipe sensor DHT22 (sensor putih)
10 #define DHTTYPE DHT22
11 DHT dht(dhtPin, DHTTYPE);
12
13 void setup() {
14   Serial.begin(115200);
15   dht.begin();
16
17   // Konfigurasi pin aktuator
18 }
19
20 void loop() {
21   // Baca sensor LDR
22   int ldrValue = analogRead(ldrPin);
23
24   // Baca sensor DHT22
25   float temperature = dht.temperature();
26   float humidity = dht.humidity();
27
28   // Logika kontrol
29   if (ldrValue < 1000) {
30     digitalWrite(relayPin, HIGH);
31     digitalWrite(ledPin, HIGH);
32   } else {
33     digitalWrite(relayPin, LOW);
34     digitalWrite(ledPin, LOW);
35   }
36
37   // Tampilkan data ke Serial Monitor
38   Serial.print("Cahaya (ADC): ");
39   Serial.print(ldrValue);
40   Serial.print(" | Suhu: ");
41   Serial.print(temperature);
42   Serial.println(" °C");
43
44   // Kirim pesan status
45   Serial.println(">>> STATUS: PERINGATAN! Kondisi Buruk -> Relay & LED AKTIF!");
46 }

Output Serial Monitor X
Message (Enter to send message to 'NodeMCU 1.0 (ESP-12E Module)' on 'COM3')
New Line 115200 baud

-----
Cahaya (ADC): 1024 | Suhu: 32.20 °C
>>> STATUS: PERINGATAN! Kondisi Buruk -> Relay & LED AKTIF!
-----
Cahaya (ADC): 1024 | Suhu: 32.10 °C
>>> STATUS: PERINGATAN! Kondisi Buruk -> Relay & LED AKTIF!
-----

Ln 71, Col 2 NodeMCU 1.0 (ESP-12E Module) on COM3
```

Penjelasan:

Tugas 2.7 ini bertujuan membangun sistem kontrol otomatisasi gudang (*Smart Warehouse*) menggunakan NodeMCU ESP8266. Sistem membaca input dari sensor **DHT** (suhu) dan **LDR** (cahaya) untuk mengontrol **Relay** dan **LED**. Menggunakan **operator logika OR (||)**, aktuator akan otomatis menyala jika salah satu kondisi terpenuhi, yaitu suhu di atas 34°C atau kondisi ruangan gelap ($ADC < 300$). Pin data DHT dipasang pada **D5** dan Relay dikontrol menggunakan logika **Active-Low**.